

Password Vault Project

Complete Documentation Manual

Comprehensive Guide on Client-Side PBKDF2 Key Derivation, AES-256-GCM Zero-Knowledge Encryption, Security Audit & All Project Features

Project Title: Password Vault / Vault Management System

Architecture: Zero-Knowledge Client-Side Encryption (Web Crypto API + Spring Boot)

Encryption Standards: PBKDF2 (SHA-256, 100,000 iterations), AES-256-GCM (96-bit IV)

Backend Tech Stack: Java 17, Spring Boot 3, Spring Security, JWT, JPA, Hibernate, H2/PostgreSQL

Frontend Tech Stack: React, Vite, Tailwind CSS, Lucide Icons, SweetAlert2

Key Security Features: Password Generator, Real-time Health Audit, Encrypted Backup/Restore, Auto-Lock Idle Inactivity, OTP Multi-Factor Authentication

1. Executive Summary & Architecture

The Password Fault / Vault Project is an enterprise-grade, zero-knowledge credential management system designed to eliminate plaintext password storage across servers, network traffic, and persistent databases. The system enforces client-side cryptographic isolation, ensuring that plain text passwords, secure notes, credit cards, and personal identities are encrypted on the client device using the Web Crypto API prior to transmission.

Core Architecture Principles:

- Zero-Knowledge Security Model: The backend application server and database never receive, process, or store unencrypted passwords or the user's master key.
- Client-Side Key Derivation: Master passwords derive 256-bit symmetric AES keys using PBKDF2 with 100,000 iterations and cryptographic salt.
- Authenticated Symmetric Encryption: Galois/Counter Mode (AES-256-GCM) ensures confidentiality and payload integrity verification.
- Full Feature Suite: Includes password generation, real-time security auditing, auto-rotation, encrypted JSON/CSV backup import & export, idle auto-lock timeout, and OTP verification.

Security Aspect	Traditional Password Vault	Password Fault Project (Zero-Knowledge)
Password Encryption	Server-side DB encryption / Plaintext in transit	Client-Side AES-256-GCM via Web Crypto API
Server Knowledge	Server has master key or access to secrets	Zero-Knowledge; server stores only ciphertext & IV
Key Derivation	Static key or simple MD5/SHA1 hashing	PBKDF2 SHA-256 with 100,000 iterations + Salt
Session Management	Standard long-lived session cookie	JWT + 1-Min Idle Inactivity Auto-Lock & Key Purge
Health Auditing	Manual or missing	Automated scanner with 1-Click Auto-Rotation

2. Cryptographic Infrastructure & Key Derivation

The core cryptographic engine is implemented in JavaScript via frontend/src/utlis/crypto.js using standard W3C Web Crypto API calls (window.crypto.subtle), eliminating external vulnerable encryption dependencies.

2.1 Master Key Derivation (PBKDF2 SHA-256)

When a user logs in, their Master Password and account Salt (Base64) are processed through Password-Based Key Derivation Function 2 (PBKDF2):

```
// PBKDF2 AES-256 Key Derivation Implementation (frontend/src/utils/crypto.js)
export async function deriveKey(masterPassword, saltBase64) {
  const enc = new TextEncoder();
  const passwordBuffer = enc.encode(masterPassword);
  const binarySalt = atob(saltBase64);
  const saltBuffer = new Uint8Array(binarySalt.length);
  for (let i = 0; i < binarySalt.length; i++) {
    saltBuffer[i] = binarySalt.charCodeAt(i);
  }
  const baseKey = await window.crypto.subtle.importKey(
    'raw', passwordBuffer, { name: 'PBKDF2' }, false, ['deriveKey']
  );
  return await window.crypto.subtle.deriveKey(
    { name: 'PBKDF2', salt: saltBuffer, iterations: 100000, hash: 'SHA-256' },
    baseKey, { name: 'AES-GCM', length: 256 }, false, ['encrypt', 'decrypt']
  );
}
```

2.2 AES-256-GCM Encryption & Decryption Mechanics

Each secret item generates a fresh, non-repeating 96-bit (12-byte) Initialization Vector (IV) using `window.crypto.getRandomValues`. GCM mode guarantees both confidentiality and authentication tags preventing tampering.

```
// AES-256-GCM Encryption (frontend/src/utils/crypto.js)
export async function encryptPassword(plainText, key) {
  const enc = new TextEncoder();
  const iv = window.crypto.getRandomValues(new Uint8Array(12)); // 96-bit IV
  const encryptedBuffer = await window.crypto.subtle.encrypt(
    { name: 'AES-GCM', iv }, key, enc.encode(plainText)
  );
  return {
    encryptedPassword: btoa(String.fromCharCode(...new Uint8Array(encryptedBuffer))),
    iv: btoa(String.fromCharCode(...iv))
  };
}
```

3. Step-by-Step Guide: How to Create & Encrypt Passwords

This section provides an explicit end-to-end operational guide detailing how credentials are created, generated, encrypted client-side, transmitted, stored in the backend database, and subsequently retrieved and decrypted.

Step 1: User Authentication & Key Generation

- 1. User submits login or registration credentials (Username/Email & Password).
- 2. Backend verifies account hash via Spring Security PasswordEncoder (BCrypt).
- 3. System returns JWT Session Token and account salt.
- 4. Frontend immediately executes `deriveKey(masterPassword, salt)` to derive the 256-bit AES master key in memory.

Step 2: Password Creation & Generation

- 1. User clicks Add Item or opens the Password Generator Modal.
- 2. User can specify password parameters: Length (8–64 characters), Uppercase (A-Z), Lowercase (a-z), Numbers (0-9), and Symbols (!@#%\$%^&*).
- 3. Generator executes `generateSecurePassword(length, options)` using cryptographically secure random bytes:

```
// Secure Random Password Generator (frontend/src/utils/crypto.js)
export function generateSecurePassword(length = 18, options) {
  const charset = { uppercase: 'A..Z', lowercase: 'a..z', numbers: '0..9', symbols: '!@#$$%' };
  let validChars = '';
  if (options.uppercase) validChars += charset.uppercase;
  if (options.lowercase) validChars += charset.lowercase;
  if (options.numbers) validChars += charset.numbers;
  if (options.symbols) validChars += charset.symbols;
  const array = new Uint8Array(length);
  window.crypto.getRandomValues(array); // CS-PRNG
  let result = '';
  for (let i = 0; i < length; i++) result += validChars[array[i] % validChars.length];
  return result;
}
```

Step 3: Real-Time Strength Scoring

As the user types or generates a password, `calculatePasswordStrength(password)` calculates a score from 0 to 100 based on length and character variance, labeling the result: Weak (<40), Moderate (40-69), Strong (70-89), or Very Strong (90-100).

Step 4: Client-Side Encryption & Transmission

- 1. The plain password is passed into `encryptPassword(plainText, key)`.
- 2. Function outputs `encryptedPassword` (Base64) and `iv` (Base64).
- 3. Frontend sends JSON DTO request to REST endpoint `POST /api/vault`:

```
// Payload sent over REST API to Spring Boot Backend:
{
  "title": "Corporate Email",
  "username": "user@infosys.com",
  "encryptedPassword": "q9K3xL1N/8vB...== [BASE64 CIPHERTEXT]",
  "iv": "u8F3xP0aM1b...== [BASE64 96-BIT IV]",
  "url": "https://mail.infosys.com",
  "category": "LOGIN",
  "favorite": true,
  "notes": "Primary work email account"
}
```

Step 5 & 6: Backend Storage & Decryption on Demand

Spring Boot VaultService.java maps the payload into a VaultItem JPA Entity associated with the authenticated User entity. When credentials are downloaded via GET /api/vault, items remain encrypted until the user explicitly toggles password visibility or copies the secret.

4. Comprehensive Project Feature Documentation

The Password Fault Project incorporates a complete set of security modules designed to manage the entire lifecycle of credentials, backup safety, and session protection.

Feature Component	Technical Description & Implementation	Security Benefit
1. Password Generator Modal	Generates 8-64 char cryptographically random strings using CS-PRNG (crypto.getRandomValues) with configurable options.	Prevents weak human-created passwords.
2. Real-Time Strength Meter	Evaluates character variance, entropy, and length rules in real time, producing a 0-100 visual score meter.	Instant feedback during password creation.
3. Security Audit Scanner	Scans all vault items for weak passwords (score < 50) and reused passwords across accounts.	Identifies high-risk credential hygiene.
4. 1-Click Auto-Rotation	Allows users to rotate weak or reused passwords directly from the Audit modal with 1 click, re-encrypting automatically.	Rapid remediation of compromised items.
5. Encrypted Backup Export	Exports vault records to JSON files encrypted with a master passphrase via PBKDF2/AES-GCM or unencrypted JSON/CSV.	Safe portable offline backups.
6. Vault Import Tool	Parses JSON backup files or standard Chrome/Bitwarden CSV exports, batch-encrypting items client-side before storage.	Seamless migration from existing tools.
7. Idle Inactivity Auto-Lock	Monitors user inputs. Triggers 10s countdown modal after 1 min idle, automatically logging out and clearing RAM keys.	Protects unattended physical workstations.
8. OTP Multi-Factor Auth	Spring Boot OtpService generates 6-digit one-time passcodes sent via email for user registration and password resets.	Prevents unauthorized account takeover.
9. Category & Favorites	Organizes items into LOGIN, CARD, NOTE, IDENTITY categories with quick search and favorite pinning.	Efficient UI management.
10. Clipboard Auto-Wipe	Copies secret to clipboard with automated UI notification and clipboard clearing logic.	Prevents credential leakage via clipboard history.

5. Backend REST API & Database Schema

The Java Spring Boot backend manages user accounts, JWT security filtering, OTP verification, and relational persistence using JPA Hibernate.

HTTP Method	Endpoint Path	Description & Functionality
POST	/api/auth/send-otp	Sends 6-digit registration OTP to user email.
POST	/api/auth/verify-otp	Verifies 6-digit email registration OTP code.
POST	/api/auth/register	Registers user after OTP verification, returns JWT token.
POST	/api/auth/login	Authenticates user against BCrypt hash, returns JWT token.
POST	/api/auth/forgot-password/send-otp	Sends password reset OTP code to registered email.
POST	/api/auth/reset-password	Resets user password after OTP verification.
GET	/api/vault	Retrieves all encrypted vault items for authenticated user.
POST	/api/vault	Saves a new encrypted vault item (title, username, ciphertext, IV).
PUT	/api/vault/{id}	Updates existing encrypted vault item.
DELETE	/api/vault/{id}	Deletes vault record from database.

6. Postman REST API Testing Guide

Below are the exact Postman testing endpoints, HTTP methods, headers, and sample JSON payloads for testing authentication and password reset APIs:

1. Send OTP (POST <http://localhost:8080/api/auth/send-otp>)

```
Header: Content-Type: application/json
Body:
{
  "email": "user@gmail.com"
}
```

2. Verify OTP (POST <http://localhost:8080/api/auth/verify-otp>)

```
Header: Content-Type: application/json
Body:
{
  "email": "user@gmail.com",
  "otp": "123456"
}
```

3. Register (POST <http://localhost:8080/api/auth/register>)

```
Header: Content-Type: application/json
Body:
{
  "fullName": "Rabindra Dakua",
  "username": "09Rabindra",
  "email": "user@gmail.com",
  "password": "Password123!"
}
```

4. Login (POST <http://localhost:8080/api/auth/login>)

```
Header: Content-Type: application/json
Body:
{
  "email": "user@gmail.com",
  "password": "Password123!"
}
```

5. Reset Password (POST <http://localhost:8080/api/auth/reset-password>)

```
Header: Content-Type: application/json
Body:
{
  "email": "user@gmail.com",
  "otp": "123456",
  "newPassword": "NewPassword123!"
}
```

7. System Verification & Approval

DOCUMENTATION APPROVAL & SYSTEM VERIFICATION (JAVA GENERATED)

Project Title: Password Vault Management System

Status: Fully Functional & Verified

Cryptographic Engine: PBKDF2 (SHA-256) + AES-256-GCM (Web Crypto API)

Backend: Spring Boot 3 REST API with Security & JWT

Frontend: React + Vite + Tailwind CSS

Report Date: August 2026